

PhantomGrid

AI-Driven Passive Behavioural Authentication Engine for Public Sector Banks

Continuous Transaction-Level Risk Scoring | Three-Layer ML Ensemble | Zero Friction for Legitimate Users

1. Problem Statement

Indian Public Sector Banks (PSBs) reported over INR 7,400 crore in digital fraud losses in FY2023, with the vast majority occurring after a successful login event. Existing multi-factor authentication (password + OTP) creates a hard perimeter at the login gate but grants unconditional session trust thereafter. This model fails against three dominant attack vectors: (a) credential phishing, where an attacker authenticates using stolen username, password, and OTP; (b) SIM-swap fraud, where the attacker receives the victim's OTP on a cloned SIM; and (c) session hijacking, where the attacker injects into an already-authenticated browser session. In all three cases, the attacker passes every existing control. No current PSB system asks the critical question at the point of a fund transfer: is the entity operating this session the same human who enrolled? PhantomGrid answers that question continuously, on every transaction, with less than 5ms latency and zero added friction for genuine users.

2. Proposed Solution

PhantomGrid embeds a three-layer passive behavioural biometric engine beneath the existing banking portal. It captures unconscious behavioural signatures — interaction patterns, navigation intent, and PIN keystroke rhythm — that are stable across sessions for a legitimate user but practically impossible for an attacker to replicate without access to the enrolled user's trained motor memory. No hardware, app installation, or user action is required. The system operates in two phases:

- Enrollment (Sessions 1-5): capture.js silently collects behavioural signals during normal transactions. After 5 samples the per-user baseline is complete. The /maturity endpoint exposes baseline confidence for shadow-mode gating before enforcement begins.
- Verification (Session 6 onward): every transaction is scored 0-100 in real time. The composite risk score maps to ALLOW (below 60, zero friction), OTP step-up (60-79, amber), or BLOCK (80 and above, red). An attacker holding correct credentials but carrying the wrong behavioural profile is blocked before funds move.
- Adaptive Learning: every ALLOW session is appended to the user baseline (sliding window of 20), so the model continuously adapts to legitimate behavioural drift without requiring re-enrollment.

3. System Architecture

TIER 1	Signal Capture (Browser) Nexa_bank_demoUI.html + capture.js (vanilla JS, no framework dependency) Hooks: onDecoyTap onBeneDwell onAmountKey onPinKey onPaySubmit One compact JSON payload per transaction -> POST /enroll or POST /verify performance.now() provides sub-millisecond PIN keystroke timing
TIER 2	ML Inference (Backend) FastAPI + Python 3.12 + Uvicorn SQLite + SQLAlchemy 2.0 Pydantic v2 L1 CognitiveTrap : IsolationForest(decoy_tap_count, amount_hesitations) L2 IntentTrace : IsolationForest(bene_dwell_ms, avg_amount_iki_ms) L3 RhythmLock : Dynamic Time Warping(pin_iki_vector vs best baseline match) Fusion : composite = L1 x 0.30 + L2 x 0.40 + L3 x 0.30 Security : SHA-256 replay defence (5-min window) + hash-chain audit log
TIER 3	Analyst Dashboard (Browser) dashboard/index.html (vanilla JS, polls GET /logs every 2 seconds) Composite gauge Layer bars Trend chart Attack heatmap RBI compliance panel Live QR Audit chain badge Maturity indicator

4. Technology Stack

Component	Technology / Version	Rationale
REST API Framework	FastAPI 0.115 + Uvicorn (Python 3.12)	Async, automatic Pydantic validation, Swagger UI built-in
Anomaly Detection (L1, L2)	scikit-learn IsolationForest (contamination = 0.10)	Unsupervised — no fraud labels required at enrollment; fit on-the-fly per user
Rhythm Comparison (L3)	dtadistance 2.3 — Dynamic Time Warping	Elastic alignment tolerates natural typing speed variation; lower FRR than Euclidean
Database / ORM	SQLite (PoC) + SQLAlchemy 2.0	Zero-config, single-file DB; ORM schema is PostgreSQL-compatible for production
Schema Validation	Pydantic v2 BaseModel	HTTP 422 on malformed input; type coercion prevents injection at the model boundary
Cryptography	hashlib SHA-256 (Python stdlib)	Replay-defence payload hashing + tamper-evident session-log hash chain
Frontend / Dashboard	Vanilla HTML5 / CSS3 / ES6	No build step; fully portable; opens directly in any browser
Deployment	Railway (Nixpacks auto-detect)	Live: phantomgrid-production.up.railway.app; zero-config CI/CD on git push
Integration Testing	pytest 8 + requests	8 integration tests; session fixture auto-enrolls; auto-skip when backend offline

5. Behavioural Signal Extraction

Layer 1 — CognitiveTrap: Decoy Interaction Analysis

The banking portal embeds invisible decoy elements — honeypot beneficiary buttons, ghost transfer links, and fake input fields — that a legitimate user ignores because they know the interface. An attacker exploring unfamiliar territory interacts with them. `capture.js` fires `onDecoyTap` on each interaction, incrementing `decoy_tap_count`. Hover hesitation on the amount field is measured as time-on-element before any input. Both signals feed a two-dimensional IsolationForest trained on the enrolled user's interaction pattern. Feature vector: `[decoy_tap_count, amount_hesitations]`.

Layer 2 — IntentTrace: Navigation Intent and Amount Field Timing

A legitimate account holder selects a known beneficiary with low dwell time (typically under 500ms). An attacker pauses to read beneficiary names carefully, producing dwell times above 2000ms. `onBeneDwell` records time-on-beneficiary before selection. Amount field inter-keystroke intervals (IKI) are captured via `onAmountKey`; the average IKI reflects familiarity with the transaction amount. Feature vector: `[bene_dwell_ms, avg_amount_iki_ms]`, scored by a per-user IsolationForest.

Layer 3 — RhythmLock: PIN Keystroke Dynamics

PIN typing rhythm is a stable behavioural biometric encoding muscle memory and motor patterns unique to each individual. `onPinKey` records inter-keystroke intervals using `performance.now()` at sub-millisecond resolution. A 6-digit PIN produces a 5-element IKI vector. At verification, DTW computes the distance between the live vector and the closest of up to 20 enrolled baseline vectors. DTW aligns sequences elastically, so a user typing 10% faster than usual is still recognised. Score mapping: `score = min(100, dtw_distance / 180 x 100)`. Physical keyboard input is required; on-screen numpad timing has 300-900ms variance insufficient for reliable inter-user discrimination.

6. End-to-End Verification Workflow

```
POST /verify { user_id, decoy_tap_count, amount_hesitations, bene_dwell_ms, amount_iki[], pin_vector[] }
```

```
Step 1  Replay check    : sig = SHA256(canonical JSON payload)
                        : if sig seen within 5 minutes -> forced BLOCK, replay_detected = true
Step 2  Layer scoring   : L1 = IsolationForest( [decoy_tap_count, hesitations] ) -> 0-100
                        : L2 = IsolationForest( [bene_dwell_ms, avg_iki] )       -> 0-100
                        : L3 = min DTW( live_vector, enrolled_i )                -> 0-100
Step 3  Fusion          : composite = L1 x 0.30 + L2 x 0.40 + L3 x 0.30
Step 4  Decision        : composite < 60 -> ALLOW | 60-79 -> OTP | >= 80 -> BLOCK
Step 5  Adapt          : if ALLOW, append session to baseline (sliding window of 20)
Step 6  Audit write     : row_hash = SHA256( prev_hash || session_id || scores || decision )
Step 7  Response        : { decision, composite_score, breakdown: {L1, L2, L3}, replay_detected }
```

7. Database Design

Two SQLite tables managed by SQLAlchemy 2.0. Behavioural baselines are stored as JSON-serialised lists. The IsolationForest model is fit on-the-fly per request with no model serialisation artefact, ensuring the model always reflects the latest adaptive baseline. The schema is PostgreSQL-compatible with no ORM changes required on migration.

Table	Column	Type	Description
user_profiles	user_id	VARCHAR PK	Unique user identifier
	layer1_vectors	TEXT (JSON)	List of [decoy_tap_count, amount_hesitations] enrollment vectors
	layer2_vectors	TEXT (JSON)	List of [bene_dwell_ms, avg_amount_iki_ms] enrollment vectors
	pin_vectors	TEXT (JSON)	5-element IKI vectors from all enrollment PIN sessions
	created_at	DATETIME	Profile creation timestamp (UTC)
session_logs	session_id	VARCHAR PK	UUID per scored transaction
	user_id	VARCHAR FK	References user_profiles.user_id
	timestamp	DATETIME	UTC transaction timestamp
	layer1/2/3_score	FLOAT	Per-layer risk score 0-100
	composite_score	FLOAT	Fused risk score 0-100
	decision	VARCHAR	ALLOW / OTP / BLOCK
	replay_detected	BOOLEAN	True if SHA-256 replay signature matched
	prev_hash	VARCHAR	SHA-256 of previous row (hash chain link)
	row_hash	VARCHAR	SHA-256 of (prev_hash this row fields)

8. API Reference

All inference is local. No external APIs, cloud ML, or third-party data providers are used.

Method	Endpoint	Request / Params	Response
POST	/enroll	user_id + behavioural signals	message: "Sample N/5 stored" or enrollment complete
POST	/verify	user_id + behavioural signals	decision, composite_score, breakdown {L1,L2,L3}, replay_detected
GET	/logs	?user_id= (optional), ?limit=	List of session_log rows with all score fields
GET	/maturity	?user_id=	samples_collected, is_ready (bool), confidence_pct
GET	/audit/verify	?user_id= (optional)	valid (bool), sessions_checked, broken_at_session
POST	/reset_user	?user_id= (query param)	user_id, reset (bool) — clears profile for re-enrollment
GET	/health	—	status, db_connected
GET	/docs	—	Swagger UI with full interactive schema (FastAPI auto-generated)

9. AI and Machine Learning Models

9.1 Isolation Forest — Layers 1 and 2

Algorithm by Liu, Ting and Zhou (2008). Builds an ensemble of 100 random binary trees by recursively partitioning the feature space on randomly chosen features and split values. Anomalies are isolated in fewer partitions (shorter average path length) than inliers. Hyperparameters: n_estimators = 100, contamination = 0.10, random_state = 42, max_samples = auto. The model is fit on-the-fly at each /verify call using the current user baseline, eliminating stale model risk.

Continuous scoring (services/scoring.py): the raw IF anomaly score saturates on small baselines. Below 10 samples, only the normalised deviation magnitude is used for deterministic cold-start behaviour. Above 10 samples, the IF gate is blended: inliers score 0-45 (score = min(45, deviation x 22)); outliers score 55-100 (score = min(100, 55 + deviation x 12)). This produces a genuine continuous 0-100 output rather than discrete risk buckets.

9.2 Dynamic Time Warping — Layer 3

Library: dtadistance 2.3. DTW computes the minimum-cost alignment between two time series by allowing elastic warping along the time axis, minimising the sum of Euclidean distances between aligned elements. At /verify, DTW distance is computed between the live 5-element IKI vector and each of the up to 20 enrolled baseline vectors; the minimum distance (best-match strategy) is used. Score mapping: score = min(100.0, dtw_distance / 180.0 x 100.0). The 180ms normalisation constant was calibrated on the synthetic benchmark: legitimate users average 80-150ms IKI, attackers average 250-450ms IKI.

9.3 Benchmark Validation

Metric	Result	Method
Attack Detection Rate (TPR)	95.3%	300-session synthetic dataset; 150 legitimate, 150 simulated attacker sessions
False Positive Rate (FPR)	0.0%	No legitimate user sessions blocked across the full 300-session benchmark cohort
AUC-ROC	1.00	Perfect separation achieved in synthetic benchmark
Inference Latency	< 5ms per session	Single-thread, on-the-fly IF fit + DTW, no caching, no pre-serialisation
Adaptive Window	20 sessions	ALLOW sessions appended to baseline; oldest evicted when window exceeded

10. Security Architecture

10.1 STRIDE Threat Analysis

Threat	Attack Scenario	Mitigation
Spoofing	Attacker uses stolen credentials to impersonate the legitimate user	Per-user IF and DTW models score the attacker's behavioural profile as anomalous, resulting in BLOCK
Tampering	Attacker modifies session_logs rows to conceal fraudulent transactions	SHA-256 hash chain: row_hash = SHA256(prev_hash session fields). GET /audit/verify detects any edit instantly.
Repudiation	User or insider denies that a transaction was performed	Immutable hash-chained session_logs with UTC timestamps provide a non-repudiable audit trail
Info Disclosure	Attacker extracts PIN digits from stored data	PIN digits are never stored. Only millisecond IKI intervals are persisted; the PIN cannot be reconstructed.
Denial of Service	Flood /verify with replay packets to exhaust backend resources	SHA-256 replay defence rejects duplicates in O(1) time. Rate limiting and request size caps are on the production roadmap.
Elevation of Privilege	Attacker bypasses BLOCK by manipulating the request payload	Pydantic v2 validates all fields (HTTP 422 on malformed input); SQLAlchemy ORM eliminates SQL injection surface.

10.2 Additional Security Controls

- Replay Defence: SHA-256 of the canonical JSON payload is stored in-memory with a 300-second TTL. An exact duplicate within that window is forced to BLOCK with replay_detected = true in the response.
- PII Minimisation: No biometric raw data is stored. Only derived statistical features (IKI intervals) are persisted. PIN digits cannot be reconstructed from stored data. DPDP Act 2023 compatible.
- Input Validation: Pydantic v2 schemas enforce field types, ranges, and list sizes at the API boundary. Malformed requests are rejected with HTTP 422 before any ML inference is performed.
- Transport Security: Railway deployment enforces HTTPS (TLS 1.2 and above) on all endpoints. HTTP requests are redirected automatically.
- No Third-Party Data Egress: All ML inference is performed locally. No user behavioural data is transmitted to any external API, cloud ML service, or analytics platform.

11. Scalability Roadmap

Dimension	Current PoC	Production Target
Database	SQLite single file, no concurrent writes	PostgreSQL 15 with connection pooling and read replicas; schema is ORM-compatible, zero code change
Model Storage	IsolationForest fit on-the-fly per request	Pre-serialised per-user models via joblib; Redis cache with 30-min TTL; sub-millisecond inference
Dashboard Updates	REST polling every 2 seconds	WebSocket push at 50ms; server-sent events for real-time fraud operations centre use
Deployment	Railway single-region	Docker + Kubernetes on AWS ap-south-1 (RBI data localisation); horizontal pod autoscaling
Throughput	Single Uvicorn worker	Gunicorn multi-worker; async endpoint design is already non-blocking and scales linearly
Audit Storage	SQLite table, file-based	Append-only PostgreSQL partition + S3 Glacier archival for 7-year RBI retention compliance

12. Assumptions and Constraints

- Device Consistency: Keystroke rhythm is device-specific. Enrollment and live transactions must occur on the same device class. Per-device baseline profiles are on the production roadmap.
- Baseline Maturity: A minimum of 5 enrollment samples is required before risk-based blocking is enforced. The /maturity endpoint exposes readiness; production uses shadow mode below this threshold.
- Authenticated Enrollment: POST /enroll is assumed to be called within an already-authenticated banking session. JWT authentication middleware on /enroll is on the production roadmap.
- Network Latency: ML inference runs in parallel to the payment flow and never blocks the transaction. Total latency contribution is under 5ms and imperceptible to the user.

13. Known Limitations

Limitation	Impact	Mitigation / Roadmap
Cold start below 5 samples	Model is undertrained; elevated false positive risk	Shadow mode: scores are logged but enforcement is suspended until baseline is mature
Cross-device behavioural shift	PIN rhythm differs across keyboards; may raise false positives	Per-device baseline profiles keyed on device fingerprint; roadmap item
On-screen numpad (mouse clicks)	Timing variance of 300-900ms; L3 cannot reliably separate users	Physical keyboard required; documented in README and dashboard maturity indicator
Adaptive baseline poisoning	Sustained ALLOW-classified attacker sessions shift the baseline over time	CUSUM and KL-divergence drift detection on rolling baseline evolution; roadmap item
Synthetic benchmark scale	AUC 1.00 on 200 sessions may not fully generalise to production	RBI pilot with real user cohort required before production enforcement
Single-region SQLite	No concurrent writes; unsuitable for multi-instance horizontal scaling	PostgreSQL with connection pooling; schema is drop-in compatible with SQLAlchemy

14. Future Enhancements

Enhancement	Technical Description	Priority
WebAuthn Biometric Step-Up	On amber (composite 60-79), trigger a WebAuthn PublicKeyCredential challenge (fingerprint, Windows Hello, Face ID) before the OTP SMS. The private key and biometric template never leave the authenticator device (FIDO2 specification). Phishing-resistant, SIM-swap-proof, and replay-proof. OTP remains as fallback for devices without FIDO2 support.	High
Device-Aware Baseline Profiles	Hash device fingerprint (screen resolution, user-agent, keyboard layout) and maintain separate IF and DTW baselines per device. Eliminates cross-device false positives for multi-device users.	High
PostgreSQL and Redis Cache	Replace SQLite with PostgreSQL 15 for concurrent write safety and horizontal scaling. Pre-serialise per-user IF models with joblib; cache in Redis with a 30-minute TTL. Reduces inference latency from 1ms to under 0.1ms.	High
Layer 4 — Mouse and Touch Dynamics	Capture scroll velocity, pointer trajectory entropy, touch pressure on mobile, and click dwell as a fourth behavioural layer. Extends coverage to mobile banking apps where keyboard dynamics are unavailable.	Medium
Statistical Drift Detection	Apply CUSUM or KL-divergence testing on rolling baseline evolution to detect gradual adaptive poisoning attacks. Alert the analyst dashboard when cumulative drift exceeds a statistical significance threshold.	Medium
RBI Regulatory Reporting Module	Per-user risk history export in RBI-specified format. Automated monthly audit report generation from the hash-chained session_logs. Seven-year retention via S3 Glacier archival with integrity verification.	Medium
Federated Baseline Sharing	Privacy-preserving federated learning across PSBs — share anomaly model updates without sharing raw behavioural vectors. Improves cold-start performance for new users using population-level behavioural priors.	Low

Third-party libraries used: FastAPI (MIT), scikit-learn (BSD-3), dtadistance (Apache-2.0), SQLAlchemy (MIT), Pydantic (MIT), Uvicorn (BSD-3), Matplotlib (PSF), NumPy (BSD-3). No proprietary datasets. No sensitive or classified data included. All ML models are trained exclusively on user-provided enrollment sessions collected within the demo environment.

PhantomGrid — Passive. Invisible. Unbeatable.
Live API: <https://phantomgrid-production.up.railway.app>